



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

DISTRIBUTED SYSTEMS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Architecture

TOTAL: 10 QUESTIONS

Q1 What is Distributed Systems and what problem in Architecture does it solve?

Q6 How does Distributed Systems handle reliability and high availability?

Q2 What are the core components or concepts in Distributed Systems?

Q7 What observability does Distributed Systems provide out of the box?

Q3 How does Distributed Systems compare to its main alternatives?

Q8 What are common performance bottlenecks in Distributed Systems?

Q4 How do you get started with Distributed Systems for a new project?

Q9 What security best practices apply when running Distributed Systems?

Q5 What configuration options most affect Distributed Systems' behavior in production?

Q10 What mistakes do teams commonly make when adopting Distributed Systems?



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Distributed Systems is a tool or platform

Mitigation

Distributed Systems is a tool or platform that automates or simplifies a specific concern in the Architecture domain, reducing manual effort and operational complexity for engineering teams.

A6 Distributed Systems provides HA through leader election, replication, or stateless

availability

Distributed Systems provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 Distributed Systems has key building blocks that

Primitives

Distributed Systems has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Distributed Systems exposes metrics (Prometheus endpoint)

monitoring

Distributed Systems exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 Distributed Systems trades off simplicity against feature hardening

hardening

Distributed Systems trades off simplicity against feature richness differently than its alternatives. Choose Distributed Systems when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured

performance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install Distributed Systems via its package manager

Gotchas

Install Distributed Systems via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run Distributed Systems with the principle of

Assessment

Run Distributed Systems with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency

concurrency

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A10 Common pitfalls include skipping the documentation and

documentation

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.